

TunePilot (v1.0)

Complete Reference Manual: Predefined Functions & CLI Commands

1. Overview & System Capabilities

TunePilot is an autonomous LLM fine-tuning orchestrator with a hybrid interactive terminal chatbot UI and scriptable CLI. It inspects datasets, discovers and ranks candidate base models, calculates hardware feasibility, plans experiments, generates training configurations, submits jobs to remote GPU backends (Kaggle Dual T4s with 30h/week free quota), and selects the winning checkpoint.

2. Interactive Chatbot Commands & Prompts

To open the chatbot, type **tunepilot** in your terminal. It features conversational memory, pronoun resolution, and multi-modal attachments.

Goal	Example Prompt	Action Performed
Analyze Data	Analyze ./examples/sample_dataset.json	Profiles schema, tokens, task format, anomalies, and leakage
Discover Models	Find the best 7B models for this task	Searches catalogs, filters hardware, displays ranked candidates
Context Resolution	Why was that second model rejected?	Resolves pronoun/ordinal references and explains architectural choices
Attachments	@docs/guide.md Use these hyperparameters	Chunks document with overlap and injects context into prompts
Launch Training	Train top candidate on Kaggle T4x2	Generates plan, renders Safety Card, and asks for [Y/n] confirmation
Job Status	What is the status of my jobs?	Renders live table with Job IDs, models, statuses, progress, and costs
Model Export	Export winning model to GGUF	Merges LoRA weights into base model and outputs standard GGUF file

3. CLI Subcommands Reference

tunepilot doctor

Runs environment diagnostics on Python libraries, CUDA GPU, VRAM, and Kaggle API credentials.

```
> tunepilot doctor
```

tunepilot init --goal

Initializes project directory layout, SQLite database registry, and starter orchestrator.yaml.

```
> tunepilot init --goal
```

tunepilot dataset analyze

Analyzes local or remote dataset format, tokens, sequence lengths, and task domain.

```
> tunepilot dataset analyze
```

tunepilot models discover [--task sft]

Discovers and ranks candidate models based on task, parameters, and hardware feasibility.

```
> tuneupilot models discover [--task sft]
```

tuneupilot hardware detect

Detects local NVIDIA GPU(s), CPU cores, RAM, and lists cloud GPU backend capabilities.

```
> tuneupilot hardware detect
```

tuneupilot plan [--backend kaggle_t4x2]

Builds pilot test runs and full experiment execution plans with compute/cost estimates.

```
> tuneupilot plan [--backend kaggle_t4x2]
```

tuneupilot train --model

Generates training configuration and submits job to compute backend.

```
> tuneupilot train --model
```

tuneupilot jobs status

Monitors queued, running, and completed jobs with real-time progress % and ETA.

```
> tuneupilot jobs status
```

tuneupilot compare

Computes normalized multi-factor composite scores across checkpoints and selects winner.

```
> tuneupilot compare
```

tuneupilot export --experiment-id

Merges LoRA adapters and exports standalone FP16 or GGUF weights.

```
> tuneupilot export --experiment-id
```

4. Python SDK & Predefined Functions API

```
from orchestrator.terminal.core import TunePilotCore
core = TunePilotCore(project_name='my-project')

# 1. Analyze Dataset
report = core.analyze_dataset('./data/train.jsonl')

# 2. Discover Models
models = core.discover_models(task='sft', quality_first=True)

# 3. Launch Training
job_ids = core.launch_training(models=['Qwen/Qwen2.5-7B'], backend='kaggle_t4x2')

# 4. Check Status & ETA
jobs = core.get_jobs_status()

# 5. Compare & Pick Winner
comparison = core.compare_models()
```